

Chapitre 3 : Fonctions

TD / TP

Table des matières

1	Travaux Dirigés	2
1.1	Analyse	2
1.1.1	Programme	2
1.1.2	Programme	2
1.1.3	Programme	2
1.1.4	Programme	2
1.1.5	Programme	2
1.1.6	Programme	3
1.1.7	Programme	3
1.2	Prototypes de fonction	3
1.2.1	est_entier()	3
1.2.2	min_entiers_positifs()	3
1.2.3	pgcd()	3
1.2.4	est_present_caractere()	3
1.2.5	occurrences_caractere()	3
1.2.6	indice_caractere()	3
1.3	Définitions de fonction	4
1.3.1	somme_premiers_entiers()	4
1.3.2	decompter()	4
1.3.1	compter()	4
2	Travaux Pratiques	4
2.1	racine_carree.py	4
2.2	triangle.py	5
2.3	combinaisons.py	6
2.4	remplacer_chaine.py	6
2.5	decrire_forme.py	7
2.6	Remise	8

1 Travaux Dirigés

Vous réalisez les exercices suivants sur papier, sans utiliser l'interpréteur Python.

1.1 Analyse

Indiquez si les définitions et appels de fonctions suivants sont corrects, et proposez une correction en cas d'erreur.

1.1.1 Programme

```
def est_pair(n: int):  
    return n % 2 == 0  
  
print(est_pair(3.5))  
if est_pair(4):  
    print('OK')
```

1.1.2 Programme

```
def est_positif(n = 0: float) -> bool:  
    return n >= 0  
  
print(est_positif(-3.14))
```

1.1.3 Programme

```
def combinaisons(n: int = 0, p: int) -> int:  
    # code fonction...  
  
print(combinaisons(4, 3))  
print(combinaisons(3))  
print(combinaisons())
```

1.1.4 Programme

```
def arrangements(n: int, p: int = 1) -> int:  
    # code fonction...  
  
print(arrangements(4, 3))  
print(arrangements(3))  
print(arrangements())  
print(arrangements(4, 3, 2))
```

1.1.5 Programme

```
def est_premier(n: int) -> bool:  
    # code fonction...  
  
print(est_premier(-3))
```

1.1.6 Programme

```
def foo(n: int):  
    if n >= 0:  
        return n ** 2  
    else:  
        return 'hi !'
```

```
p = foo(-1) + foo(1)
```

1.1.7 Programme

```
def bar(n: int) -> int:  
    print(n)  
    bar(n + 1)  
    if n == 0:  
        print(0)
```

```
bar(10)
```

1.2 Prototypes de fonction

Ecrivez sur papier le *prototype* des fonctions spécifiées dans les paragraphes ci-dessous. Le prototype d'une fonction est la première ligne de sa définition (`def ... :`), avec annotations de type.

Identifiez les erreurs que la fonction peut rencontrer, notamment concernant ses paramètres, et proposez une valeur retour spéciale par laquelle elle signalera cette erreur à l'appelant.

1.2.1 `est_entier()`

Détermine si un nombre réel est entier.

1.2.2 `min_entiers_positifs()`

Demande à l'utilisateur une suite d'entiers positifs et détermine la plus petite des valeurs saisies.

1.2.3 `pgcd()`

Détermine le Plus Grand Commun Diviseur de deux entiers.

1.2.4 `est_present_caractere()`

Détermine un caractère est présent dans une chaîne de caractères.

1.2.5 `occurrences_caractere()`

Détermine le nombre d'occurrences d'un caractère dans une chaîne de caractères.

1.2.6 `indice_caractere()`

Détermine l'indice de la première occurrence d'un caractère dans une chaîne de caractères.

1.3 Définitions de fonction

Définissez les fonctions spécifiées ci-après et montrez un exemple d'appel.

1.3.1 somme_premiers_entiers()

Retourne la somme des n premiers entiers, n étant un entier variable. La fonction est écrite de façon récursive.

1.3.2 decompter()

Affiche la séquence « n $n-1$ $n-2$... 3 2 1 partez ! » sur une seule ligne, n étant un entier variable. La fonction est écrite de façon récursive.

1.3.1 compter()

Affiche la séquence « 1 2 3 ... $n-2$ $n-1$ n go ! » sur une seule ligne, n étant un entier variable. La fonction est écrite de façon récursive.

2 Travaux Pratiques

Il s'agit de définir en Python les fonctions décrites dans les paragraphes suivants.

Le titre du paragraphe est le nom du script à remettre, le texte qui suit spécifie la fonction à définir, en donnant son prototype, son docstring et des exemples d'exécution sous l'interpréteur Python.

Vous ne devez utiliser que des constructions et fonctions que nous avons étudiées en cours. Certains exercices peuvent spécifier des contraintes de réalisation supplémentaires.

Le testeur fourni ne teste que quelques cas représentatifs. Le testeur qui établira votre note testera des cas supplémentaires et vérifiera le respect des contraintes de réalisation.

Vos fonctions doivent tester la validité des *valeurs* de leurs paramètres, mais pas celle de leur *types* : le testeur passera toujours des arguments conformes en type aux paramètres.

2.1 racine_carree.py

```
def racine_carree(nombre: float, n: int = 100) -> float:
    """ Calcule la racine carrée d'un réel par une suite récurrente.
```

Suite utilisée :

$$x_0 = E(a)$$

$$x_n = (x_{n-1} + (a / x_{n-1})) / 2, \text{ pour } n > 0$$

Parametres :

nombre : nombre objet du calcul (supérieur ou égal à 0).

n : indice du terme de la suite à calculer.

Retourne :

Le terme d'indice n de la suite récurrente, ou -1.0 si erreur.

"""

Exemples :

```
>>> racine_carree(2)
1.414213562373095
>>> racine_carree(2, 0)
2.0
>>> racine_carree(2, 2)
1.4166666666666665
>>> racine_carree(4)
2.0
>>> racine_carree(-3.14)
-1.0
>>>
```

2.2 triangle.py

```
def triangle(hauteur: int, est_creux: bool = True, \
orientation: int = 0) -> str:
    """ Représente un triangle à l'aide de 'X'.
```

Parametres :

```
    hauteur : hauteur du triangle (supérieure à 1).
    est_creux : True si seul le périmètre est représenté.
    orientation : orientation du triangle (0 : bas, 1 : droite ou
                2 : haut).
```

Retourne :

```
    La chaine représentant le triangle, ou '' si erreur.
    """
```

Exemples :

```
>>> print(triangle(5))
X
XX
X X
X  X
XXXXX
>>> print(triangle(4, False, 1))
X
XX
XXX
XXXX
>>>
>>>
```

```
>>> print(triangle(3, True, 2))
XXX
 XX
  X
>>>
```

2.3 combinaisons.py

```
def combinaisons(n: int, p: int) -> int:
    """ Calcule  $C_n^p$ , le nombre de combinaisons de p éléments parmi n.

    Parametres :
        n : positif ou nul.
        p : compris entre 0 et n inclus.

    Retourne :
        Le nombre de combinaisons, ou -1 si erreur.
    """
```

Exemples :

```
>>> combinaisons(5, 5)
1
>>> combinaisons(5, 3)
10
>>> combinaisons(3, 4)
-1
>>>
```

Contraintes :

La fonction est implémentée de façon récursive, à partir de la définition inductive suivante de C_n^p :

$$C_n^0 = 1, \text{ pour } n \geq 0$$

$$C_n^p = (n / p) C_{n-1}^{p-1}, \text{ pour } n \geq 0 \text{ et } n \geq p > 1$$

2.4 remplacer_chaine.py

```
def remplacer_chaine(s1: str, s2: str, s3: str) -> str:
    """ Remplace les occurrences d'une chaîne de caractères par une
        autre.

    Parametres :
        s1 : chaîne de caractères dans laquelle remplacer.
        s2 : chaîne de caractères à remplacer.
        s3 : chaîne de caractères de remplacement.
```

Retourne :

La chaîne résultant des remplacements.
 """

Exemples :

```
>>> remplacer_chaine('bonjour', 'xyz', '')
'bonjour'
>>> remplacer_chaine('haaaaa !', 'aa', 'o')
'hooa !'
>>> remplacer_chaine('salut', 'salut', '')
''
>>>
```

2.5 decrire_forme.py

```
def decrire_forme(forme: str) -> (str, (int, int), bool):
    """ Détermine les caractéristiques d'une forme.
```

La fonction reconnaît les triangles rectangles isocèles, les rectangles et les carrés, détermine leurs dimensions, et si elles sont creuses ou non.

La forme est constituée d'un seul et même caractère, quelconque. Si ses dimensions sont trop petites pour déterminer si elle est creuse ou non, elle est déclarée non creuse par défaut.

Parametres :

forme : chaîne de caractères représentant une forme.

Retourne :

Un tuple (type, (hauteur, largeur), est_creuse) décrivant la forme, ou None si la forme est incorrecte ou non reconnue.
 """

Exemples :

```
>>> decrire_forme(triangle(5, True, 1))
('triangle', (5, 5), True)
>>> decrire_forme('===\n===\n===')
('carre', (3, 3), False)
>>> decrire_forme('000\n0 0\n000')
('carre', (3, 3), True)
>>> decrire_forme('%%\n%\n%\n%')
('carre', (3, 3), False)
>>> decrire_forme('XXX\nXYX\nXXX')
>>>
```

2.6 Remise

Remettez svp vos fichiers .py dans l'espace Moodle prévu à cet effet.

Vous devez les remettre à *plat*, c'est-à-dire sans répertoires ou sous-répertoires intermédiaires et sans les regrouper en archive. Il vous suffit de faire un simple glisser-déposer des différents fichiers depuis votre répertoire de développement dans l'espace Moodle.

Vous pouvez remettre vos fichiers autant de fois que vous le voulez jusqu'à la date limite. Toute nouvelle version doit *écraser* la précédente : les fichiers nommés par exemple `script(1).py` seront ignorés lors de la correction.